RAPORT PROIECT

Sistem de Management al Interventiilor
INTERVENTION360

Aplicatie Web cu Spring Boot si Thymeleaf

Alexandrescu Anamaria - 331AB

12 ianuarie 2026

Cuprins

1 Introducere **2**
1.1 Prezentare Generala . . . 2
1.1.1 Obiective . . . 2
1.1.2 Tehnologii Utilizate . . . 2

2 Functionalitati **4**
2.1 Functionalitati de Baza . . . 4
2.1.1 Sistem Dual de Autentificare . . . 4
2.1.2 Inregistrare si Validari Java . . . 4
2.1.3 Board si Cautare . . . 5
2.1.4 Management CRUD Interventii . . . 5
2.1.5 Dashboard si Self-Service Client . . . 5
2.1.6 Facturare si Statistici . . . 6
2.2 Functionalitati Suplimentare . . . 7
2.2.1 Dark Mode cu Persistenta . . . 7
2.2.2 Design Responsive . . . 7
2.2.3 Flash Messages . . . 7
2.2.4 Session Management si Securitate . . . 8

3 Arhitectura Aplicatiei **9**
3.1 Structura in Straturi . . . 9
3.1.1 Model Layer - Entitati . . . 9
3.1.2 Repository Layer . . . 10
3.1.3 Service Layer . . . 10
3.1.4 Controller Layer . . . 11
3.1.5 View Layer . . . 15

4 Testarea Aplicatiei **16**
4.1 Verificarea Autentificarii si Securitatii . . . 16
4.2 Functionarea Interventiilor si a Datelor . . . 18
4.3 Interfata si Experienta Utilizatorului . . . 18

5 Imbunatatiri Posibile **19**

Capitolul 1

Introducere

1.1 Prezentare Generala

INTERVENTION360 este o aplicatie web dezvoltata pentru gestionarea completa a interventiilor tehnice si serviciilor de mentenanta. Sistemul a fost conceput pentru a raspunde nevoilor companiilor care ofera servicii tehnice la domiciliu, precum instalatii electrice sau reparatii generale.

Aplicatia ofera functionalitati distincte pentru doua categorii de utilizatori: angajati ai companiei si clienti. Fiecare categorie beneficiaza de o interfata personalizata care raspunde nevoilor specifice si ofera doar functionalitatile relevante pentru rolul respectiv.

Sistemul centralizeaza intregul proces de management al interventiilor.

1.1.1 Obiective

- **Digitalizarea completa** a proceselor de interventie, eliminand documentele fizice si centralizand toate informatiile intr-o baza de date accesibila in timp real.

- **Imbunatatirea comunicarii** dintre companie si clienti prin oferirea accesului direct la informatii despre interventiile lor, fara necesitatea de apeluri telefonice etc.

- **Transparenta** in statusul lucrarilor, permitand atat angajatilor cat si clientilor sa vada in ce stadiu se afla fiecare interventie.

- **Automatizarea generarii** de rapoarte si facturi, reducand timpul pentru taskuri administrative.

1.1.2 Tehnologii Utilizate

Aplicatia este construita pe o stiva tehnologica moderna, asigurand performanta si scalabilitate:

Backend:

- **Spring Boot 3.x:** Framework Java pentru dezvoltare rapida si configurare automata.
- **Spring Data JPA & Hibernate:** Pentru maparea obiectelor pe baza de date.
- **Microsoft SQL Server:** Sistem de gestiune a bazelor de date relationale pentru stocarea informatiilor.

Frontend:

- **Thymeleaf:** Motor de sabloane server-side pentru randarea dinamica a paginilor HTML.
- **HTML, CSS, JavaScript:** Tehnologii fundamentale pentru structura, stil si interactivitate.
- **Chart.js:** Biblioteca pentru generarea graficelor de analiza in modulul de statistici.
- **UI/UX:** Design responsive pentru compatibilitate cu dispozitive mobile si suport pentru Dark Mode.

Capitolul 2

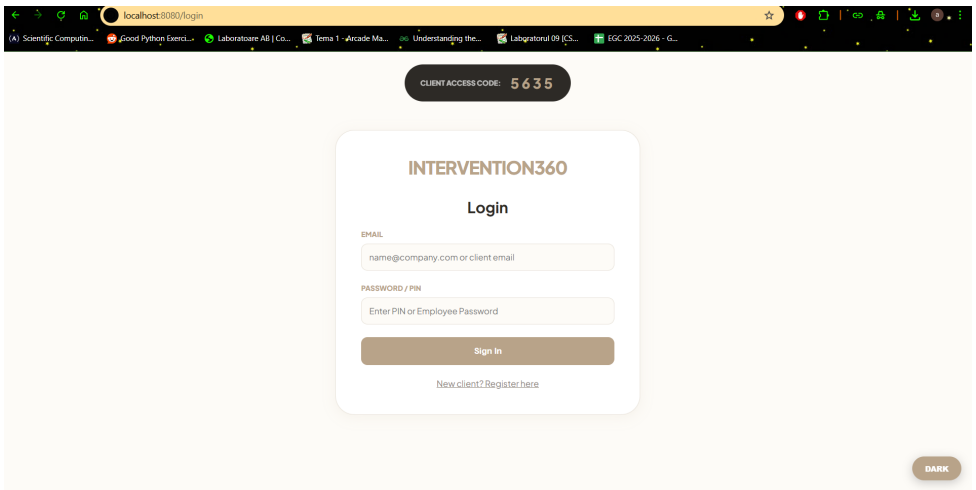
Functionalitati

2.1 Functionalitati de Baza

2.1.1 Sistem Dual de Autentificare

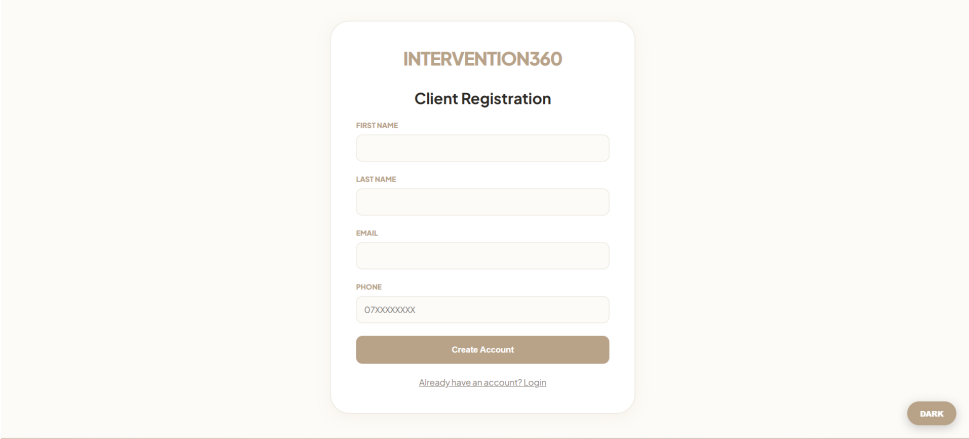
Aplicatia foloseste un sistem de login adaptat pentru doua tipuri de utilizatori:

- **Angajati:** Autentificare cu email corporativ (@firma.ro) si parola, oferind acces total la management, Kanban board si actiuni administrative.
- **Clienti:** Autentificare simplificata cu email personal si un PIN de 4 cifre generat automat, oferind acces la dashboard-ul personal (status interventii, facturi, profil).



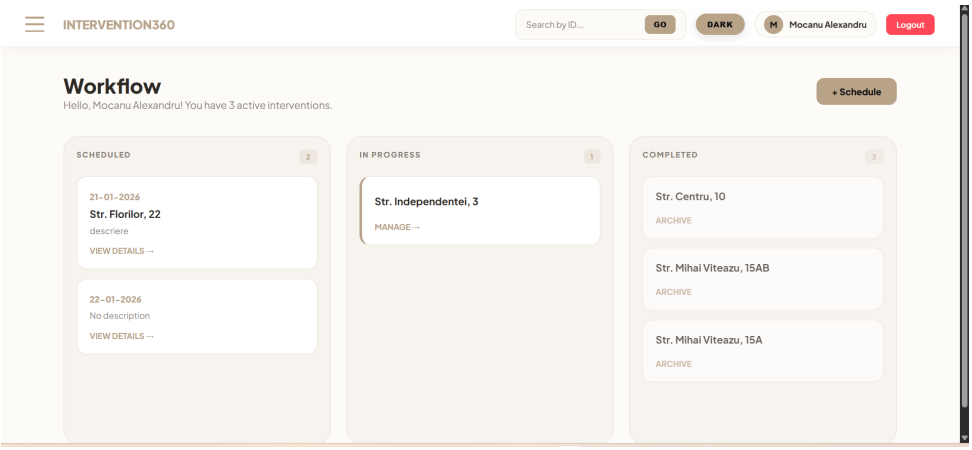
2.1.2 Inregistrare si Validari Java

Clientii se pot inregistra singuri, sistemul efectuand validari manuale in Java pentru nume (doar litere), telefon (format RO, 10 cifre) si email (structura corecta, fara domenii corporative). Dupa inregistrare, utilizatorul beneficiaza de auto-login instantaneu.



2.1.3 Board si Cautare

Managementul interventiilor se realizeaza printr-un board cu trei coloane: *Programate*, *In Desfasurare* si *Finalizate*. Functia de cautare este implementata prin procesare liniara in Java, permitand identificarea rapida a unei interventii dupa ID-ul numeric, cu validare de tip try-catch pentru siguranta datelor.



2.1.4 Management CRUD Interventii

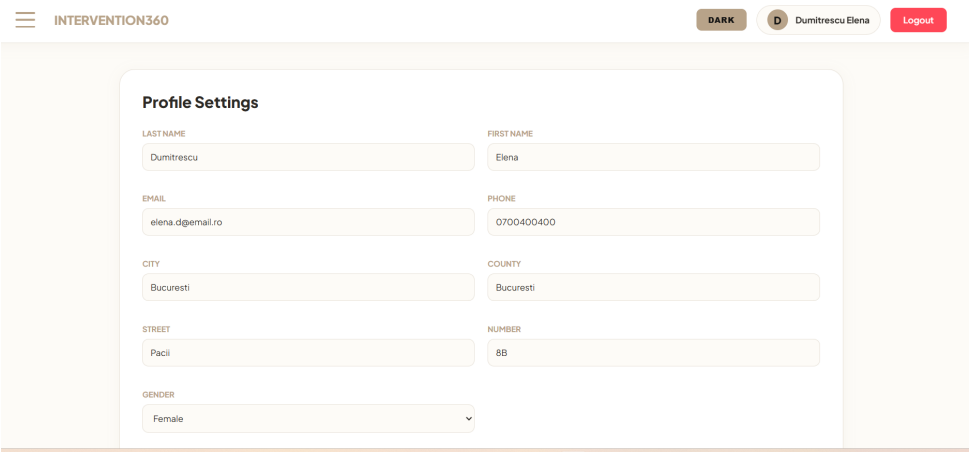
Sistemul permite gestionarea fluxului de lucru:

- **Creare:** Selectare client, serviciu si data prin datetime picker.
- **Actualizare:** Schimbarea statusului muta automat cardul in coloana corespunzatoare.
- **Stergere:** Proces cu cleanup in cascada (sterge automat alocarile si materialele asociate).

2.1.5 Dashboard si Self-Service Client

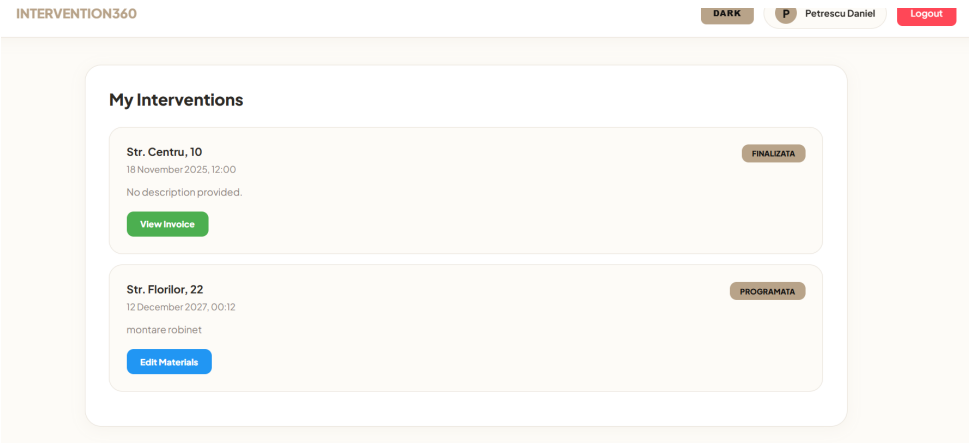
Clientii au la dispozitie un portal dedicat unde pot:

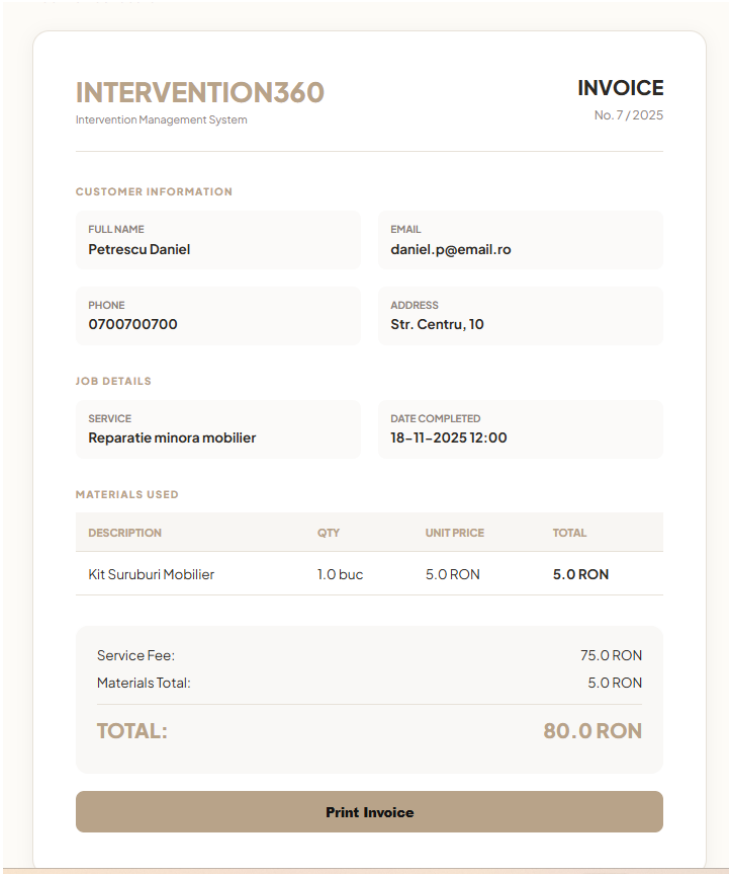
- Modifica datele de profil (adresa).
- Programara singuri interventii noi.
- Alege sau modifica materialele dorite pentru interventiile programate inainte de sosirea tehnicianului.



2.1.6 Facturare si Statistici

Pentru interventiile finalizate, sistemul genereaza automat facturi profesionale optimizate pentru printare, calculand automat subtotalurile materialelor si taxa de serviciu. Managerii pot accesa un modul de statistici cu grafice interactive (Chart.js) care afiseaza distributia interventiilor si topul celor mai profitabile servicii sau clienti.





2.2 Functionalitati Suplimentare

2.2.1 Dark Mode cu Persistenta

Aplicatia include un mod intunecat implementat prin variabile CSS si JavaScript. Acesta ofera un confort vizual sporit si reduce consumul de energie pe dispozitivele mobile. Preferinta utilizatorului este salvata in `localStorage`, asigurand persistenta temei la reincarcarea paginii sau la vizite ulterioare.

2.2.2 Design Responsive

Interfata este optimizata pentru orice tip de dispozitiv (desktop, tableta, smartphone). Pe ecranele mici, meniul principal se transforma intr-un tip hamburger, iar elementele complexe precum Board-ul sau tabelele de date isi adapteaza layout-ul pentru a ramane lizibile si usor de utilizat prin atingere.

2.2.3 Flash Messages

Pentru a imbunatati experienta utilizatorului (UX), sistemul ofera feedback instant prin mesaje temporare. Acestea semnaleaza erori de validare. Mesajele sunt transmise prin `RedirectAttributes` si dispar automat dupa vizualizare.

2.2.4 Session Management si Securitate

Securitatea datelor este asigurata prin gestionarea sesiunilor HTTP:

- **Separarea rolurilor:** Accesul este restrictionat strict pe baza tipului de utilizator (Angajat vs. Client).
- **Validarea proprietatii:** Un client poate accesa sau edita doar interventiile care ii sunt asociate direct.
- **Logout securizat:** Invalidarea completa a sesiunii la deconectare pentru a preveni accesul neautorizat.

Capitolul 3

Arhitectura Aplicatiei

3.1 Structura in Straturi

Aplicatia urmeaza arhitectura MVC (Model-View-Controller) stratificata:

3.1.1 Model Layer - Entitati

Client

- Atribute: id, nume, prenume, email, telefon, adresa completa, sex
- Metoda helper: getNumeComplet()

User (Angajat)

- Atribute: date personale + CNP, calificare, salariu orar, supervizor
- Folosit pentru autentificare angajati

Interventie

- Atribute: id, clientId, serviciuId, data, adresa, status, descriere
- Trei statusuri posibile: Programata, In desfasurare, Finalizata

Serviciu

- Atribute: id, denumire, pretStandard

Material

- Atribute: id, denumire, pretAchizitie, unitateMasura

InterventieMateriale

- Tabel junction pentru relatia many-to-many
- Composite key: interventieId + materialId
- Atribut suplimentar: cantitateUtilizata

3.1.2 Repository Layer

ClientRepository:

- findMaxId() - pentru generare ID nou
- findByEmail() - pentru autentificare
- findClientiCuStatistici()

```
// gaseste client după email
@Query("SELECT c FROM Client c WHERE c.email = :email")  2 usages
Optional<Client> findByEmail(String email);

// clienti cu statistici
@Query(value = """  2 usages
    SELECT C.client_id, C.nume, C.prenume, C.email, COUNT(I.interventie_id) as nr_lucrari
    FROM dbo.clienti C
    LEFT JOIN dbo.interventii I ON C.client_id = I.client_id
    GROUP BY C.client_id, C.nume, C.prenume, C.email
    ORDER BY nr_lucrari DESC
""", nativeQuery = true)
List<Map<String, Object>> findClientiCuStatistici();
```

InterventieRepository:

- findByStatus() - pentru columns
- findByClientId() - pentru dashboard client
- deleteAlocariAngajati() - cleanup pentru foreign keys

InterventieMaterialeRepository:

- insertMaterial() - native INSERT
- deleteByInterventieId() - cleanup materiale

```
@Modifying  3 usages
@Query(value = "DELETE FROM dbo.interventie_materiale WHERE interventie_id = :interventieId", nativeQuery = true)
void deleteByInterventieId(@Param("interventieId") Integer interventieId);

@Modifying  2 usages
@Query(value = "INSERT INTO dbo.interventie_materiale (interventie_id, material_id, cantitate_utilizata) " +
        "VALUES (:interventieId, :materialId, :cantitate)", nativeQuery = true)
void insertMaterial(@Param("interventieId") Integer interventieId,
                    @Param("materialId") Integer materialId,
                    @Param("cantitate") Integer cantitate);
```

3.1.3 Service Layer

AuthService

- Gestioneaza autentificarea angajatilor
- Verificare parola universala
- Cautare utilizator in baza de date

```java
@Service  3 usages
public class AuthService {

    @Autowired  1 usage
    private UserRepository userRepository;

    private static final String UNIVERSAL_PASSWORD = "Admin123!";  1 usage

    public AuthResult authenticate(String email, String password) {  1 usage
        // vf dacă parola este cea universala
        if (!password.equals(UNIVERSAL_PASSWORD)) {
            return new AuthResult( success: false,  message: "Incorrect employee password!",  user: null);
        }

        // cauta angajatul email
        User user = userRepository.findByEmail(email).orElse( other: null);

        if (user == null) {
            return new AuthResult( success: false,  message: "Email not registered in the system. Only employees can access.",  user: null);
        }

        return new AuthResult( success: true,  message: "Authentication successful", user);
    }
```

```java
    public static class AuthResult {  5 usages
        private boolean success;  2 usages
        private String message;  2 usages
        private User user;  2 usages

        public AuthResult(boolean success, String message, User user) {  3 usages
            this.success = success;
            this.message = message;
            this.user = user;
        }

        public boolean isSuccess() { return success; }  1 usage
        public String getMessage() { return message; }  no usages
        public User getUser() { return user; }
    }
}
```

3.1.4 Controller Layer

LoginController

- GET /login - afisare pagina cu generare PIN
- POST /login - procesare autentificare duala
- POST /register - inregistrare client cu validari
- GET /logout - invalidare sesiune
- Functii auxiliare de validare

```
@PostMapping("/register")  no usages
public String register(@RequestParam String nume,
                       @RequestParam String prenume,
                       @RequestParam String email,
                       @RequestParam String telefon,
                       HttpSession session,
                       RedirectAttributes redirectAttributes) {

    List<String> errors = new ArrayList<>();

    // validare prenume
    String prenumeError = validateName(prenume, fieldName: "First Name");
    if (prenumeError != null) {
        errors.add(prenumeError);
    }

    // vf nume
    String numeError = validateName(nume, fieldName: "Last Name");
    if (numeError != null) {
        errors.add(numeError);
    }
```

```
    // daca sunt erori se returneaza toate
    if (!errors.isEmpty()) {
        redirectAttributes.addFlashAttribute( attributeName: "errors", errors);
        return "redirect:/login";
    }

    // generare ID
    Integer nextId = clientRepository.findMaxId() + 1;

    // salvare client
    Client c = new Client();
    c.setId(nextId);
    c.setNume(nume);
    c.setPrenume(prenume);
    c.setEmail(email);
    c.setTelefon(telefon);
    clientRepository.save(c);

    // auto login
    session.setAttribute( s: "client", c);
    session.setAttribute( s: "userType", o: "client");
```

```java
 /**
  nume, prenume: doar litere, minim 2 caractere
  return mesaj de eroare sau null dacă e valid
 */
private String validateName(String text, String fieldName) {  2 usages
    if (text == null || text.trim().isEmpty()) {
        return fieldName + " is required";
    }

    String trimmed = text.trim();

    if (trimmed.length() < 2) {
        return fieldName + " must have at least 2 letters";
    }

    for (int i = 0; i < trimmed.length(); i++) {
        char c = trimmed.charAt(i);
        if (!Character.isLetter(c)) {
            return fieldName + " must contain only letters";
        }
    }

    return null;
}
```

InterventieController

- GET /dashboard - board pentru angajati
- POST /search-intervention - cautare cu procesare Java
- GET/POST /adauga_interventie - creare interventie
- GET /interventie/{id} - detalii complete
- POST /update_status - schimbare status
- GET /sterge_interventie/{id} - delete
- GET /clienti - lista clienti cu statistici
- GET/POST /adaugare_client - management clienti

```java
public class InterventieController {
    @PostMapping("/search-intervention")  no usages
    public String searchIntervention(@RequestParam String interventionId,
                                     HttpSession session,
                                     RedirectAttributes redirectAttributes) {
        if (!checkAuth(session)) return "redirect:/login";

        List<String> errors = new java.util.ArrayList<>();

        // vf id gol
        if (interventionId == null || interventionId.trim().isEmpty()) {
            errors.add("Please enter an Intervention ID");
            redirectAttributes.addFlashAttribute( attributeName: "errors", errors);
            return "redirect:/dashboard";
        }

        String trimmedId = interventionId.trim();

        // vf id contine doar cifre
        boolean isNumeric = true;
        for (int i = 0; i < trimmedId.length(); i++) {
            char c = trimmedId.charAt(i);
            if (!Character.isDigit(c)) {
                isNumeric = false;
                break;
            }
        }

        if (!isNumeric) {
            errors.add("Intervention ID must contain only numbers");
            redirectAttributes.addFlashAttribute( attributeName: "errors", errors);
```

```java
// String -> Integer
Integer id;
try {
    id = Integer.parseInt(trimmedId);
} catch (NumberFormatException e) {
    errors.add("Invalid Intervention ID format");
    redirectAttributes.addFlashAttribute( attributeName: "errors", errors);
    return "redirect:/dashboard";
}

Interventie interventie = null;
List<Interventie> toateInterventiile = interventieRepository.findAll();

for (Interventie i : toateInterventiile) {
    if (i.getId().equals(id)) {
        interventie = i;
        break;
    }
}

if (interventie == null) {
    errors.add("Intervention with ID " + id + " not found");
    redirectAttributes.addFlashAttribute( attributeName: "errors", errors);
    return "redirect:/dashboard";
}

return "redirect:/interventie/" + id;
```

ClientDashboardController

- GET /client/dashboard - dashboard personalizat client
- POST /client/update-profile - editare profil
- POST /client/schedule-intervention - programare noua
- GET /client/edit-materials/{id} - editare materiale
- POST /client/save-materials - salvare modificari
- GET /client/invoice/{id} - generare factura

StatisticiController

- GET /statistici - pagina cu grafice si tabele
- Agregare date din multiple repository-uri
- Pregatire date pentru Chart.js

3.1.5 View Layer

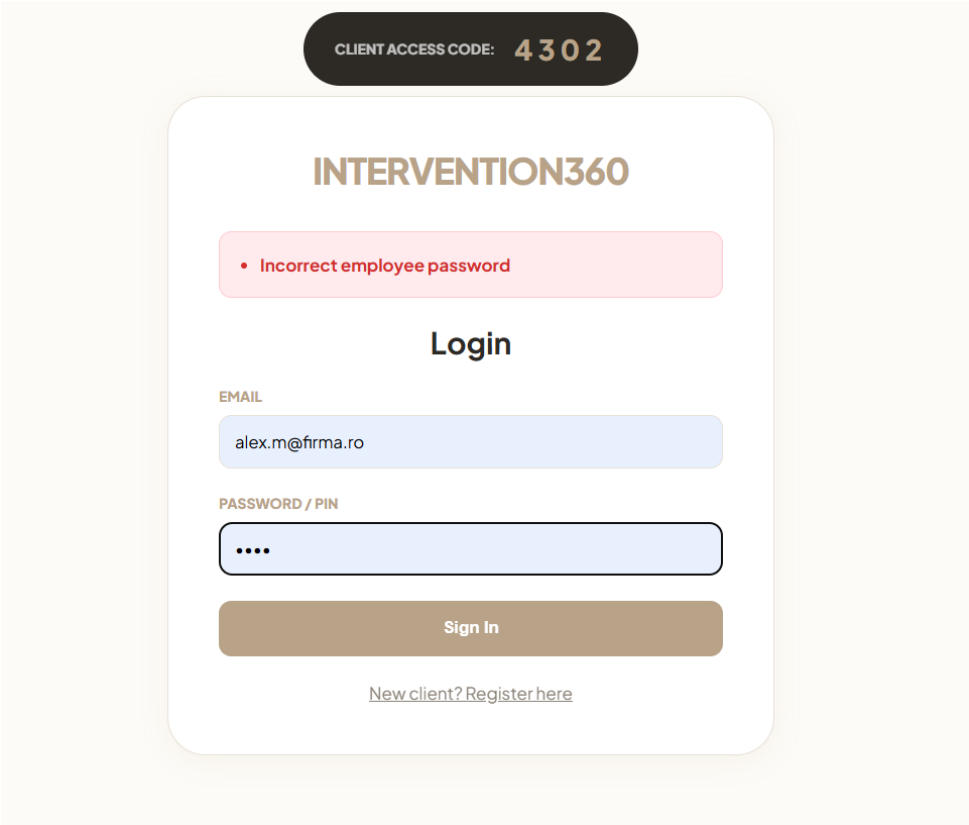
Template-uri Thymeleaf principale: `login.html`, `dashboard.html`, `client_dashboard.html`, `detalii.html`, `edit_materials.html`, `invoice.html`, `statistici.html`.

Capitolul 4

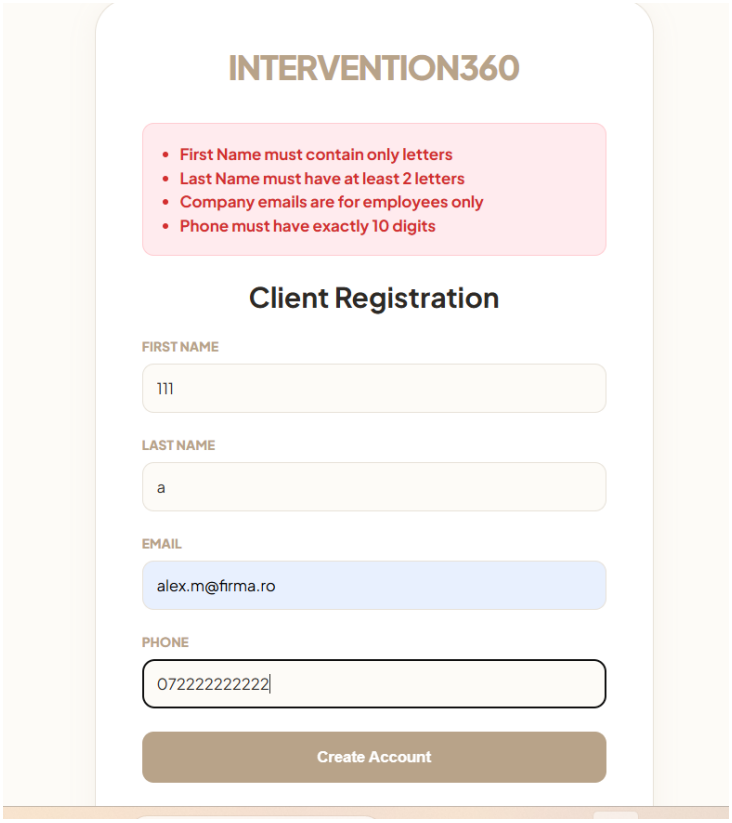
Testarea Aplicatiei

4.1 Verificarea Autentificarii si Securitatii

Testele au confirmat functionarea corecta a accesului diferentiat. Angajatii si clientii sunt directionati corect catre interfetele proprii, iar sistemul de securitate blocheaza accesul neautorizat intre conturi. Formularul de inregistrare valideaza riguros datele de intrare, respingand numerele de telefon si adresele de email care nu respecta formatul impus.

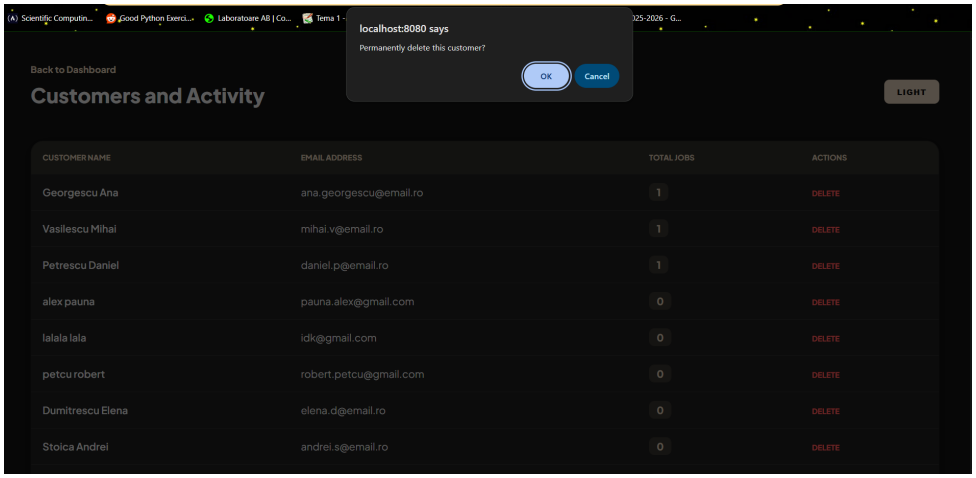


CLIENT ACCESS CODE: 4302
INTERVENTION360
Incorrect PIN
Login
EMAIL
petcu.robert@gmail.com
PASSWORD / PIN
Sign In
New client? Register here


INTERVENTION360
First Name must contain only letters
Last Name must have at least 2 letters
Company emails are for employees only
Phone must have exactly 10 digits
Client Registration
FIRST NAME
111
LAST NAME
a
EMAIL
alex.m@firma.ro
PHONE
072222222222
Create Account

4.2 Functionarea Interventiilor si a Datelor

Cardurile se muta corect in board-ul la schimbarea statusului, iar calculele pentru materiale si facturi sunt exacte. Functia de stergere a fost testata pentru a asigura eliminarea completa a datelor asociate din baza de date, prevenind aparitia erorilor de integritate. Cautarea dupa ID functioneaza rapid si gestioneaza corect eventualele erori de tastare ale utilizatorului.



4.3 Interfata si Experienta Utilizatorului

Aplicatia a fost testata pe diverse rezolutii, confirmand ca designul este relativ adaptabil atat pentru desktop, cat si pentru dispozitive mobile. Modul intunecat isi pastreaza starea intre sesiuni prin utilizarea `localStorage`. Interfata grafica a fost verificata pe browsere populare (Chrome, Firefox, Edge).

Capitolul 5

Imbunatatiri Posibile

- **Notificari Real-Time:** Implementare WebSocket pentru alerte instantanee.
- **Sistem Feedback:** Mecanism de rating si recenzii pentru servicii.
- **MFA:** Securitate in doi pasi cu cod OTP (Google Authenticator) pentru accesul angajatilor.
- **AI Support:** Chatbot inteligent pentru raspunsuri rapide la intrebari frecvente.
- **Plati Online:** Integrare cu procesatori de plati (Stripe/Netopia) pentru achitarea facturilor direct din aplicatie.